PYTHON ACTIVITY-BASED LEARNING

11 Case Studies: Flowcharts, Pseudocode, Programs, Test Cases and Output Screenshots

Student Name	Janni Chandra Sahithi
Branch	CSM
Roll Number	26001146
Repository	PythonActivity-Based-Learning

Case Studies Included

University Scholarship Allocation

Progressive Electricity Billing

Hospital Billing and Patient Classification

Hackathon Team Formation and Ranking

Food Waste Monitoring System

Multi-Branch Expense Matrix Analysis

Multi-Stage Scholarship Shortlisting

Integrated University Fee and Eligibility Audit

Smart Hostel Resource Monitoring

Multi-Route Courier Dispatch Planning

Campus Festival Registration and Financial Audit

Each case includes problem statement, objectives, assumptions, algorithm, flowchart, pseudocode, Python source code, test cases, sample output, output screenshot and conclusion.

Note: Rates, thresholds and policy rules are sample assumptions for educational purposes.

1. University Scholarship Allocation

1. Problem Statement

Evaluate aggregate marks, subject minimum, attendance and family income; rank eligible applicants.

2. Objectives

Check all eligibility conditions; rank eligible applicants by marks and attendance.

3. Assumptions

Marks >=75; subject minimum >=40; attendance >=75%; income <= Rs.200000. Example rules.

4. Algorithm

1. Read applicants

2. validate all criteria

3. store eligible applicants

4. sort by marks then attendance descending

5. display ranking.

5. Flowchart

START → Read applicant → Check all criteria → Eligible? yes: store / no: reject → More applicants? → Sort → Disp

6. Pseudocode

```
INPUT n; eligible=[]; repeat n times read applicant; if marks>=75 and subject>=40 and attendance>=75 and income<=200000 add applicant; sort by
```

7. Python Source Code / Program

```
n=int(input("Number of students: "))
students=[]
for i in range(n):
    name=input("Name: ")
    marks=float(input("Aggregate marks: "))
    subject=float(input("Minimum subject marks: "))
    attendance=float(input("Attendance %: "))
    income=float(input("Family income: "))
    if marks>=75 and subject>=40 and attendance>=75 and income<=200000:
        students.append([name,marks,attendance])
        print("Eligible")
    else:
        print("Not eligible")
students.sort(key=lambda x:(x[1],x[2]),reverse=True)
print("--- Ranking ---")
for rank,s in enumerate(students,1):
    print(rank,s[0],s[1],s[2])
```

8. Test Cases

Input / test	Expected result
Ravi — 85,60,90%,150000	Eligible
Kumar — 65,35,80%,100000	Not eligible

9. Sample Output

```
Number of students: 2
Name: Ravi
Aggregate marks: 85
Minimum subject marks: 60
Attendance %: 90
Family income: 150000
Eligible
Name: Sita
Aggregate marks: 92
Minimum subject marks: 70
Attendance %: 95
Family income: 180000
Eligible
--- Ranking ---
1 Sita 92.0 95.0
2 Ravi 85.0 90.0
```

10. Output Screenshot



11. Conclusion

Automatically checks scholarship criteria and creates a merit ranking.

2. Progressive Electricity Billing

1. Problem Statement

Calculate monthly bills with progressive slabs, fixed charge, surcharge and rebate; flag unusual increases.

2. Objectives

Calculate slab charges; apply surcharge and rebate; compare monthly bills.

3. Assumptions

First 100 units Rs.2/unit; next 100 Rs.3/unit; above 200 Rs.5/unit; fixed Rs.50; surcharge 5%; rebate 10% when units<=100; unusual increase >30%. Example rates.

4. Algorithm

1. Read month and units

2. calculate slab charge

3. add fixed charge and surcharge

4. apply rebate

5. store bill

6. compare with previous month.

5. Flowchart

START → Read units → Calculate slabs → Add fixed/surcharge → Units<=100? rebate → Store bill → More months? →

6. Pseudocode

```
FOR each month calculate energy by slabs; total=energy+50+energy*0.05; if units<=100 subtract 10%; compare current bill with previous bill; fl
```

7. Python Source Code / Program

```
n=int(input("Number of months: "))
bills=[]
for i in range(n):
    month=input("Month: ")
    units=int(input("Units: "))
    if units<=100: energy=units*2
    elif units<=200: energy=200+(units-100)*3
    else: energy=500+(units-200)*5
    total=energy+50+energy*0.05
    if units<=100: total*=0.90
    bills.append([month,total])
    print(month,"Bill Rs.",round(total,2))
print("--- Comparison ---")
for i,b in enumerate(bills):
    print(b[0],round(b[1],2))
    if i>0 and b[1]>bills[i-1][1]*1.30:
        print("Warning: Unusual increase")
```

8. Test Cases

Input / test	Expected result
80 units — January	Rs.196.20
120 units — February	Rs.323.00; increase warning

9. Sample Output

```
Number of months: 3
Month: January
Units: 80
January Bill Rs. 196.2
Month: February
Units: 120
February Bill Rs. 323.0
Month: March
Units: 250
March Bill Rs. 522.5
--- Comparison ---
January 196.2
February 323.0
Warning: Unusual increase
March 522.5
Warning: Unusual increase
```

10. Output Screenshot



11. Conclusion

Calculates example electricity bills and flags large increases for review.

3. Hospital Billing and Patient Classification

1. Problem Statement

Calculate consultation, diagnostic and treatment charges, apply concessions and classify the final bill.

2. Objectives

Validate charges; calculate total and discount; classify final bill.

3. Assumptions

Charges non-negative; age>=60 gets 20% discount, otherwise total>=10000 gets 10%; final <5000 Low, <10000 Medium, otherwise High. Example policy.

4. Algorithm

1. Read patient details

2. reject negative charges

3. sum charges

4. apply discount

5. classify final amount

6. display bill.

5. Flowchart

START → Read charges → Negative? reject → Sum charges → Age>=60? 20% off : total>=10000? 10% off → Classify final

6. Pseudocode

```
INPUT patient charges and age; if any charge<0 reject; total=sum; if age>=60 discount=20%, elif total>=10000 discount=10%, else 0; final=total
```

7. Python Source Code / Program

```
n=int(input("Number of patients: "))
for i in range(n):
    name=input("Name: ")
    c=float(input("Consultation: "))
    d=float(input("Diagnostics: "))
    t=float(input("Treatment: "))
    age=int(input("Age: "))
    if min(c,d,t)<0:
        print("Invalid charges")
        continue
    total=c+d+t
    if age>=60: discount=total*0.20
    elif total>=10000: discount=total*0.10
    else: discount=0
    final=total-discount
    if final<5000: category="Low Payment"
    elif final<10000: category="Medium Payment"
    else: category="High Payment"
    print(name,"Total:",total,"Discount:",discount,
          "Final:",final,"Category:",category)
```

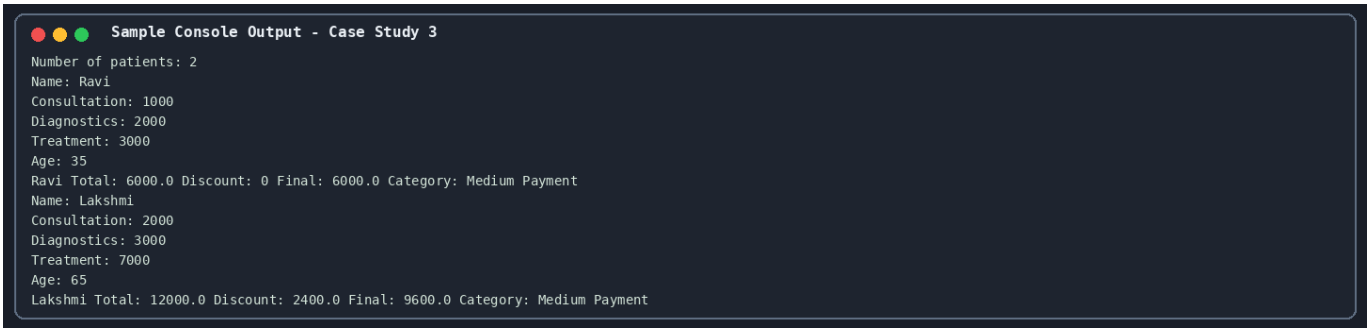
8. Test Cases

Input / test	Expected result
Ravi — 1000+2000+3000; age 35	Final 6000; Medium
Lakshmi — 2000+3000+7000; age 65	Final 9600; Medium

9. Sample Output

```
Number of patients: 2
Name: Ravi
Consultation: 1000
Diagnostics: 2000
Treatment: 3000
Age: 35
Ravi Total: 6000.0 Discount: 0 Final: 6000.0 Category: Medium Payment
Name: Lakshmi
Consultation: 2000
Diagnostics: 3000
Treatment: 7000
Age: 65
Lakshmi Total: 12000.0 Discount: 2400.0 Final: 9600.0 Category: Medium Payment
```

10. Output Screenshot



11. Conclusion

Creates patient bills and applies the stated sample discount and classification policy.

4. Hackathon Team Formation and Ranking

1. Problem Statement

Validate team sizes, total scores across rounds and rank teams with a tie-break rule.

2. Objectives

Validate team size; sum round scores; rank by total then Round 3 score.

3. Assumptions

Team size 2-4; three non-negative round scores; highest total wins; Round 3 breaks ties.

4. Algorithm

1. Read teams

2. reject invalid size

3. calculate total

4. sort by total and Round 3 descending

5. display ranks.

5. Flowchart

START → **Read team and size** → **Size valid?** → **Read round scores** → **Total** → **More teams?** → **Sort totals/tie-break** →

6. Pseudocode

```
FOR each team read name,size; if size outside 2..4 reject; read r1,r2,r3; total=sum; store; sort by total then r3 descending; display.
```

7. Python Source Code / Program

```
n=int(input("Number of teams: "))
teams=[]
for i in range(n):
    name=input("Team name: ")
    size=int(input("Team size: "))
    if size<2 or size>4:
        print("Invalid team size")
        continue
    r1=int(input("Round 1 score: "))
    r2=int(input("Round 2 score: "))
    r3=int(input("Round 3 score: "))
    teams.append([name,size,r1,r2,r3,r1+r2+r3])
teams.sort(key=lambda x:(x[5],x[4]),reverse=True)
for rank,t in enumerate(teams,1):
    print("Rank",rank,t[0],"Total",t[5],"Round 3",t[4])
```

8. Test Cases

Input / test	Expected result
Alpha — Size 3; scores 80,85,90	Total 255; first on tie-break
Gamma — Size 5	Rejected

9. Sample Output

```
Number of teams: 3
Team name: Alpha
Team size: 3
Round 1 score: 80
Round 2 score: 85
Round 3 score: 90
Team name: Beta
Team size: 4
Round 1 score: 90
Round 2 score: 85
Round 3 score: 80
Team name: Gamma
Team size: 5
Invalid team size
Rank 1 Alpha Total 255 Round 3 90
Rank 2 Beta Total 255 Round 3 80
```

10. Output Screenshot



11. Conclusion

Validates team composition and ranks teams using total score and a predefined tie-break.

5. Food Waste Monitoring System

1. Problem Statement

Compare prepared meals, served meals and leftovers; calculate wastage percentage and flag days above a limit.

2. Objectives

Validate counts; calculate wastage; warn when wastage exceeds 10%.

3. Assumptions

Counts non-negative; prepared=served+leftovers; limit 10%; if prepared=0, wastage is 0.

4. Algorithm

1. Read daily counts

2. reject invalid or inconsistent data

3. calculate leftovers/prepared*100

4. display warning if >10%.

5. Flowchart

START → Read counts → Valid and totals match? → Calculate waste % → Waste>10? warn → More days? → END

6. Pseudocode

```
FOR each day read prepared, served, leftover; reject negative counts; require served+leftover=prepared; waste=leftover/prepared*100; warn if w
```

7. Python Source Code / Program

```
n=int(input("Number of days: "))
for i in range(n):
    day=input("Day: ")
    prepared=int(input("Meals prepared: "))
    served=int(input("Meals served: "))
    leftover=int(input("Leftovers: "))
    if min(prepared,served,leftover)<0 or served+leftover!=prepared:
        print("Invalid meal data")
        continue
    waste=0 if prepared==0 else leftover/prepared*100
    print(day,"Wastage:",round(waste,2),"%")
    if waste>10: print("Warning: exceeds limit")
    else: print("Within limit")
```

8. Test Cases

Input / test	Expected result
Monday — 100 prepared, 90 served, 10 left	10%; within limit
Tuesday — 200,160,40	20%; warning

9. Sample Output

```
Number of days: 3
Day: Monday
Meals prepared: 100
Meals served: 90
Leftovers: 10
Monday Wastage: 10.0 %
Within limit
Day: Tuesday
Meals prepared: 200
Meals served: 160
Leftovers: 40
Tuesday Wastage: 20.0 %
Warning: exceeds limit
Day: Wednesday
Meals prepared: 150
Meals served: 140
Leftovers: 10
Wednesday Wastage: 6.67 %
Within limit
```

10. Output Screenshot



11. Conclusion

Identifies high-waste days and checks the consistency of meal records.

6. Multi-Branch Expense Matrix Analysis

1. Problem Statement

Analyse expenses across branches and categories with a nested list; calculate totals, averages and largest contributors.

2. Objectives

Calculate branch totals/averages; category totals; largest branch/category; grand total.

3. Assumptions

Three branches and categories Salary, Rent, Transport; values are sample amounts.

4. Algorithm

1. Set expense matrix

2. sum each row for branch totals

3. sum each column for category totals

4. identify maximums

5. display grand total.

5. Flowchart

START → Create matrix → Calculate row totals and averages → Calculate column totals → Find maximums → Display repo

6. Pseudocode

```
FOR each row total=sum(row), average=total/category count; FOR each column sum expenses; find max branch and category; grand total=sum(branch
```

7. Python Source Code / Program

```
branches=["Branch A","Branch B","Branch C"]
categories=["Salary","Rent","Transport"]
expenses=[[50000,15000,10000],[60000,12000,8000],[45000,18000,12000]]
totals=[]
for i,row in enumerate(expenses):
    total=sum(row); totals.append(total)
    print(branches[i],"Total",total,"Average",round(total/3,2))
ct=[]
for j,cat in enumerate(categories):
    value=sum(expenses[i][j] for i in range(len(branches)))
    ct.append(value); print(cat,"total",value)
print("Highest branch:",branches[totals.index(max(totals))])
print("Largest category:",categories[ct.index(max(ct))])
print("Grand total:",sum(totals))
```

8. Test Cases

Input / test	Expected result
Branch B — 60000,12000,8000	Total 80000; highest branch
Salary — 50000+60000+45000	Total 155000; largest category

9. Sample Output

```
Branch A Total 75000 Average 25000.0
Branch B Total 80000 Average 26666.67
Branch C Total 75000 Average 25000.0
Salary total 155000
Rent total 45000
Transport total 30000
Highest branch: Branch B
Largest category: Salary
Grand total: 230000
```

10. Output Screenshot



11. Conclusion

Uses nested lists to compare branch and category spending and identify the largest expense contributors.

7. Multi-Stage Scholarship Shortlisting

1. Problem Statement

Apply academic, attendance and income requirements, rank eligible students and select candidates within a fixed quota.

2. Objectives

Check criteria; rank eligible students; select no more than quota.

3. Assumptions

Marks>=75; attendance>=75%; income<=Rs.200000; rank by marks then attendance; sample rules.

4. Algorithm

1. Read quota and applicants

2. retain eligible students

3. sort by marks and attendance

4. display first quota students.

5. Flowchart

START → **Read quota/applicants** → **Check criteria** → **Add eligible** → **More applicants?** → **Sort** → **Select up to quota** →

6. Pseudocode

```
INPUT n,quota; collect applicants satisfying marks>=75, attendance>=75, income<=200000; sort marks and attendance descending; display first qu
```

7. Python Source Code / Program

```
n=int(input("Number of students: "))
quota=int(input("Scholarship quota: "))
students=[]
for i in range(n):
    name=input("Name: ")
    marks=float(input("Aggregate marks: "))
    attendance=float(input("Attendance %: "))
    income=float(input("Family income: "))
    if marks>=75 and attendance>=75 and income<=200000:
        students.append([name,marks,attendance])
    else: print("Not eligible")
students.sort(key=lambda x:(x[1],x[2]),reverse=True)
for rank,s in enumerate(students[:max(0,quota)],1):
    print("Rank",rank,s[0],"Marks",s[1],"Attendance",s[2])
print("Total selected:",min(len(students),max(0,quota)))
```

8. Test Cases

Input / test	Expected result
Sita — 92 marks, 95% attendance, income 180000	Eligible, ranks first
Kumar — 65 marks, 80% attendance, income 100000	Not eligible
Quota — 2	At most 2 selected

9. Sample Output

```
Number of students: 3
Scholarship quota: 2
Name: Ravi
Aggregate marks: 85
Attendance %: 90
Family income: 150000
Name: Sita
Aggregate marks: 92
Attendance %: 95
Family income: 180000
Name: Kumar
Aggregate marks: 65
Attendance %: 80
Family income: 100000
Not eligible
Rank 1 Sita Marks 92.0 Attendance 95.0
Rank 2 Ravi Marks 85.0 Attendance 90.0
Total selected: 2
```

10. Output Screenshot



11. Conclusion

Produces a merit-based shortlist while respecting the scholarship quota.

8. Integrated University Fee and Eligibility Audit

1. Problem Statement

Combine fee balance, attendance and internal marks to determine exam eligibility and list each failed condition.

2. Objectives

Identify pending fees; check attendance and marks; report all reasons for ineligibility.

3. Assumptions

Pending fee must be zero; attendance>=75%; internal marks>=40%; example rules.

4. Algorithm

1. Read each student's records

2. build a reason list for each failed condition

3. report eligible if no reasons, otherwise list them.

5. Flowchart

START → Read records → Check fee → Check attendance → Check marks → Reasons empty? → Eligible / Not eligible wit

6. Pseudocode

```
FOR each student set reasons=[]; if fee>0 add fee reason; if attendance<75 add attendance reason; if marks<40 add marks reason; display status
```

7. Python Source Code / Program

```
n=int(input("Number of students: "))
for i in range(n):
    name=input("Student name: ")
    fee=float(input("Pending fee: "))
    attendance=float(input("Attendance %: "))
    marks=float(input("Internal marks: "))
    reasons=[]
    if fee>0: reasons.append("Fee Pending")
    if attendance<75: reasons.append("Low Attendance")
    if marks<40: reasons.append("Low Internal Marks")
    print("Student:",name)
    if not reasons: print("Eligible for Examination")
    else:
        print("Not Eligible")
        for reason in reasons: print("-",reason)
```

8. Test Cases

Input / test	Expected result
Ravi — fee 0, attendance 85, marks 75	Eligible
Sita — fee 5000, attendance 65, marks 30	Three reasons listed

9. Sample Output

```
Number of students: 2
Student name: Ravi
Pending fee: 0
Attendance %: 85
Internal marks: 75
Student: Ravi
Eligible for Examination
Student name: Sita
Pending fee: 5000
Attendance %: 65
Internal marks: 30
Student: Sita
Not Eligible
- Fee Pending
- Low Attendance
- Low Internal Marks
```

10. Output Screenshot



11. Conclusion

Creates a reason-wise audit report so students can see which requirements remain outstanding.

9. Smart Hostel Resource Monitoring

1. Problem Statement

Calculate room-wise electricity and water charges, flag high consumption and compare usage by floor.

2. Objectives

Calculate charges; identify threshold violations; aggregate usage by floor; find highest electricity-use floor.

3. Assumptions

Electricity Rs.8/unit; water Rs.2/unit; warning above 100 electricity units or 50 water units; illustrative rates.

4. Algorithm

1. Read room and floor usage

2. validate non-negative values

3. calculate charges and warnings

4. aggregate by floor

5. find highest electricity use.

5. Flowchart

START → Read room usage → Negative? reject → Calculate charges → Check thresholds → Add floor totals → More room

6. Pseudocode

```
FOR each room read room,floor,electricity,water; reject negative values; charges=electricity*8 and water*2; warn over thresholds; sum by floor
```

7. Python Source Code / Program

```
n=int(input("Number of rooms: "))
floors={}
for i in range(n):
    room=input("Room number: ")
    floor=input("Floor: ")
    electricity=float(input("Electricity units: "))
    water=float(input("Water units: "))
    if electricity<0 or water<0:
        print("Invalid consumption")
        continue
    print(room,"Electricity charge",electricity*8,
          "Water charge",water*2,"Total",(electricity*8+water*2))
    if electricity>100: print("High electricity warning")
    if water>50: print("High water warning")
    if floor not in floors: floors[floor]=[0,0]
    floors[floor][0]+=electricity; floors[floor][1]+=water
for floor,use in floors.items():
    print("Floor",floor,"Electricity",use[0],"Water",use[1])
if floors:
    high=max(floors,key=lambda f:floors[f][0])
    print("Highest electricity floor:",high)
```

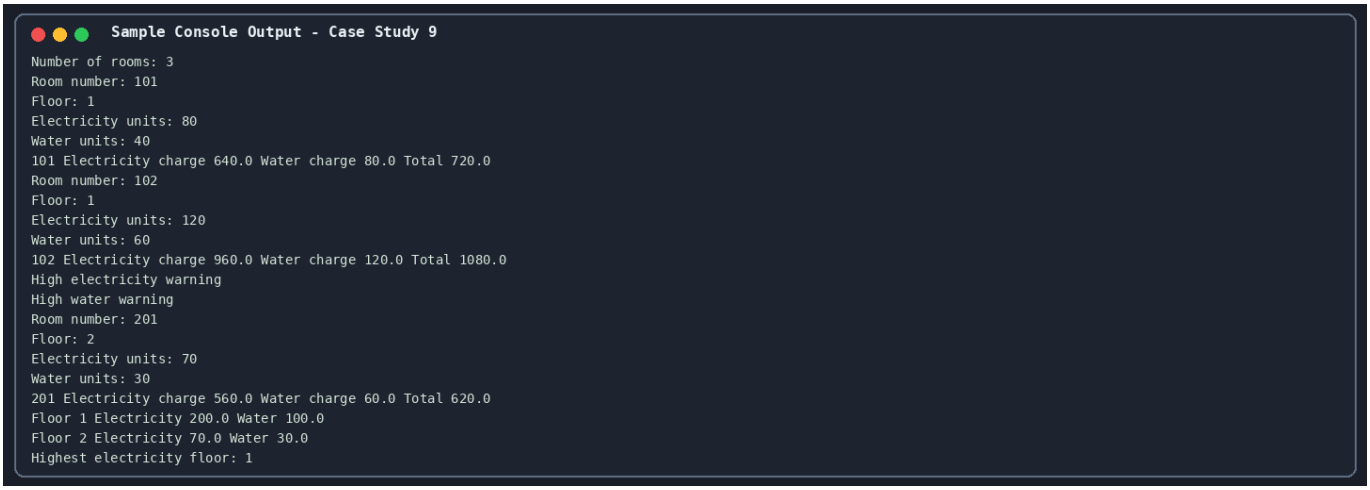
8. Test Cases

Input / test	Expected result
Room 101 — 80 electricity, 40 water	Charges 640 and 80; no warning
Room 102 — 120 electricity, 60 water	Both warnings
Floor comparison — Floor 1 total electricity 200; floor 2 total 70	Floor 1 highest

9. Sample Output

```
Number of rooms: 3
Room number: 101
Floor: 1
Electricity units: 80
Water units: 40
101 Electricity charge 640.0 Water charge 80.0 Total 720.0
Room number: 102
Floor: 1
Electricity units: 120
Water units: 60
102 Electricity charge 960.0 Water charge 120.0 Total 1080.0
High electricity warning
High water warning
Room number: 201
Floor: 2
Electricity units: 70
Water units: 30
201 Electricity charge 560.0 Water charge 60.0 Total 620.0
Floor 1 Electricity 200.0 Water 100.0
Floor 2 Electricity 70.0 Water 30.0
Highest electricity floor: 1
```

10. Output Screenshot



11. Conclusion

Helps monitor hostel consumption, calculate charges and identify high-usage floors.

10. Multi-Route Courier Dispatch Planning

1. Problem Statement

Allocate packages to vehicles using destination zone, weight, capacity and delivery priority; identify packages not dispatched.

2. Objectives

Process high-priority packages first; obey zone and capacity restrictions; report remaining capacity.

3. Assumptions

Priority 3 highest; each vehicle serves one zone; package must fit remaining capacity; simple greedy allocation.

4. Algorithm

1. Read vehicles

2. read packages

3. sort by priority descending

4. find first vehicle with matching zone and enough capacity

5. assign or report failure

6. display capacity.

5. Flowchart

START → Read vehicles and packages → Sort by priority → Find matching zone/capacity vehicle → Found? assign/reduce ca

6. Pseudocode

```
Sort packages by priority descending; for each package search vehicles; if zone matches and capacity sufficient, assign and subtract weight; d
```

7. Python Source Code / Program

```
vehicles=[]
v=int(input("Number of vehicles: "))
for i in range(v):
    name=input("Vehicle name: ")
    zone=input("Vehicle zone: ")
    capacity=float(input("Capacity kg: "))
    vehicles.append([name,zone,capacity])
n=int(input("Number of packages: "))
packages=[]
for i in range(n):
    pid=input("Package ID: ")
    zone=input("Destination zone: ")
    weight=float(input("Weight kg: "))
    priority=int(input("Priority 1-3: "))
    packages.append([pid,zone,weight,priority])
packages.sort(key=lambda p:p[3],reverse=True)
for p in packages:
    assigned=False
    for vehicle in vehicles:
        if vehicle[1]==p[1] and vehicle[2]>=p[2]:
            print(p[0],"assigned to",vehicle[0])
            vehicle[2]-=p[2]; assigned=True; break
    if not assigned: print(p[0],"cannot be dispatched")
for vehicle in vehicles:
    print(vehicle[0],"remaining capacity",vehicle[2],"kg")
```

8. Test Cases

Input / test	Expected result
P101 — Zone A, 6kg, priority 3; Van1 capacity 10	Assigned; 4kg left
P102 — Zone A, 5kg after P101	Cannot dispatch
P103 — Zone B, 4kg; Van2 capacity 8	Assigned; 4kg left

9. Sample Output

```
Number of vehicles: 2
Vehicle name: Van1
Vehicle zone: A
Capacity kg: 10
Vehicle name: Van2
Vehicle zone: B
Capacity kg: 8
Number of packages: 3
Package ID: P101
Destination zone: A
Weight kg: 6
Priority 1-3: 3
Package ID: P102
Destination zone: A
Weight kg: 5
Priority 1-3: 2
Package ID: P103
Destination zone: B
Weight kg: 4
```

```
Priority 1-3: 1
P101 assigned to Van1
P102 cannot be dispatched
P103 assigned to Van2
Van1 remaining capacity 4.0 kg
Van2 remaining capacity 4.0 kg
```

10. Output Screenshot



11. Conclusion

Allocates packages while respecting zones, capacity and priority, and identifies packages that cannot be sent.

11. Campus Festival Registration and Financial Audit

1. Problem Statement

Reconcile participant registration fees, sponsorships and expenses; detect duplicate registration IDs and calculate final balance.

2. Objectives

Reject duplicate IDs; calculate valid fees and total income; determine profit, loss or break-even.

3. Assumptions

Registration IDs must be unique; duplicate fee is not counted; balance=valid fees+sponsorship-expenses.

4. Algorithm

1. Read participants

2. reject repeated IDs

3. sum accepted fees

4. read sponsorship and expenses

5. calculate balance and display financial result.

5. Flowchart

START → Read participant → ID duplicate? reject : accept and add fee → More participants? → Read sponsorship/expenses

6. Pseudocode

```
FOR each participant: if ID exists reject; else store ID and add fee. income=fees+sponsorship; balance=income-expenses; display profit if posi
```

7. Python Source Code / Program

```
n=int(input("Number of participants: "))
participants=[]
fees=0
for i in range(n):
    reg=input("Registration ID: ")
    name=input("Name: ")
    fee=float(input("Registration fee: "))
    if any(p[0]==reg for p in participants):
        print("Duplicate registration detected")
    else:
        participants.append([reg,name]); fees+=fee
        print("Registration accepted")
sponsor=float(input("Sponsorship: "))
expenses=float(input("Festival expenses: "))
income=fees+sponsor
balance=income-expenses
print("Valid participants:",len(participants))
print("Registration fees:",fees)
print("Total income:",income)
print("Expenses:",expenses)
print("Final balance:",balance)
if balance>0: print("Profit")
elif balance<0: print("Loss")
else: print("No Profit, No Loss")
```

8. Test Cases

Input / test	Expected result
R101 first use — Fee 500	Accepted; counted
R101 duplicate — Fee 500	Rejected; not counted
Financial audit — Fees 1000, sponsorship 10000, expenses 7000	Balance Rs.4000; profit

9. Sample Output

```
Number of participants: 3
Registration ID: R101
Name: Ravi
Registration fee: 500
Registration accepted
Registration ID: R102
Name: Sita
Registration fee: 500
Registration accepted
Registration ID: R101
Name: Kumar
Registration fee: 500
Duplicate registration detected
Sponsorship: 10000
Festival expenses: 7000
Valid participants: 2
Registration fees: 1000.0
Total income: 11000.0
Expenses: 7000.0
Final balance: 4000.0
Profit
```

10. Output Screenshot



11. Conclusion

Detects duplicate registrations and provides a transparent financial summary for the festival.